

Foundations of ISTA & Sparse Coding

Notes based on discussion with JP

1 The Linear System Setup

We start with the fundamental equation of the project:

$$\mathbf{y} = \mathbf{A}\mathbf{x} \tag{1}$$

Where:

- $y \in \mathbb{R}^m$: The Observation / Measurement (Compressed Data).
- $A \in \mathbb{R}^{m \times n}$: The Dictionary / Sensing Matrix.
- $x \in \mathbb{R}^n$: The Sparse Code / Signal.

1.1 Matrix Geometry and Solutions

The nature of the solution depends on the dimensions of A :

1. Square Matrix ($m = n$):

- The system is determined.
- Unique solution: $\hat{x} = A^{-1}y$.

2. Tall Matrix ($m > n$):

- Overdetermined system (more equations than unknowns).
- Solved via Least Squares: $\hat{x} = (A^T A)^{-1} A^T y$.

3. Fat Matrix ($m < n$): (*This is our Project Context*)

- Underdetermined system.
- There are $(n - m)$ **free variables**.
- **Consequence:** Infinite solutions exist. Algebra alone cannot find the “correct” x .
- **The Fix:** We assume x is **Sparse** (K -sparsity) to pick the unique solution.
- *Visual:* y is formed by a linear combination of only a few active columns of A .

2 The Optimization Problem

Since we cannot invert a fat matrix, we frame this as a minimization problem.

2.1 The “Ideal” Problem (P0)

We want the solution with the fewest non-zeros:

$$(P0) : \min_x \|x\|_0 \quad \text{subject to} \quad \|y - Ax\|_2 \leq \epsilon \tag{2}$$

- $\|x\|_0$: The L_0 pseudo-norm (counts number of non-zeros).
- **Issue:** This is **NP-Hard**. It requires combinatorial search and is intractable for large dimensions.

2.2 The “Relaxed” Problem (P1 - LASSO)

We relax the L_0 norm to the L_1 norm (Convex Relaxation).

$$(P1) : \min_x J(x) = \min_x \left(\frac{1}{2} \|y - Ax\|_2^2 + \lambda \|x\|_1 \right) \quad (3)$$

This objective function $J(x)$ is composed of two terms: $g(x) + h(x)$.

1. Data Fidelity Term ($g(x)$):

$$g(x) = \frac{1}{2} \|y - Ax\|_2^2$$

- Ensures the solution matches the data.
- Properties: **Convex** and **Smooth** (differentiable).
- Gradient: $\nabla g(x) = A^T(Ax - y)$.

2. Sparsity Regularizer ($h(x)$):

$$h(x) = \lambda \|x\|_1$$

- Forces entries to zero.
- Properties: **Convex** but **Non-Smooth** (kinks at 0).

3 Geometric Interpretation

Why does L_1 promote sparsity while L_2 does not?

- **L2 Norm** ($\|x\|_2$): The isocontour is a **Circle**. When the circle expands to touch the solution line (subspace $y = Ax$), it usually touches at a generic point where $x_i \neq 0$.
- **L1 Norm** ($\|x\|_1$): The isocontour is a **Diamond** (Polytope). The corners of the diamond lie on the axes. The expanding diamond hits the solution line at a **corner** first. At a corner, one or more coordinates are exactly **Zero**.

4 The Algorithm: ISTA

To minimize $J(x) = g(x) + h(x)$, we use **Proximal Gradient Descent**.

The Update Step:

$$\begin{aligned} x^{t+1} &= \text{prox}_{\lambda, h}(x^t - \eta \nabla g(x^t)) \\ x^{t+1} &= \text{prox}_{\lambda, h}(x^t - \eta \nabla g(x^t)) \end{aligned} \quad (4)$$

This breaks down into two operations:

1. Gradient Descent (on Error):

$$z = x^t - \eta A^T(Ax^t - y)$$

2. Shrinkage (Soft Thresholding):

$$x^{t+1} = \mathcal{S}_\theta(z)$$

Where the Soft Thresholding operator is:

$$\mathcal{S}_\theta(z)_i = \text{sign}(z_i) \max(|z_i| - \theta, 0)$$

5 The Philosophy: Compressed Sensing vs. JPEG

To understand Compressed Sensing (CS), it is best to compare it with standard compression (JPEG). Both rely on **Sparsity**, but they use it at opposite ends of the pipeline.

5.1 1. The Old Way: JPEG Compression

Standard imaging follows a “Measure Everything, Throw Away Later” philosophy.

1. **Capture (y_{raw}):** The sensor captures **ALL** pixels (e.g., 64 pixels for an 8×8 block). The system is fully determined.
2. **Transform (x):** We compute the DCT (Discrete Cosine Transform). $x = \text{DCT}(y)$.
3. **Compress:** We observe that high-frequency coefficients in x are near-zero. We Quantize/Round them to exactly zero.
4. **Save:** We store the sparse vector x .

Critique: This is inefficient. We spent energy and time measuring 64 data points, only to discover that 59 of them were redundant. Sparsity is the *Result* of the process.

5.2 2. The New Way: Compressed Sensing

CS follows a “Measure What Matters” philosophy.

1. **Assumption:** We *assume* beforehand that the image is sparse in some domain (e.g., DCT).
2. **Capture (y):** Instead of measuring individual pixels, we measure **random linear combinations** (mixtures) of pixels.

$$y = Ax$$

Here, y is already compressed (e.g., 10 measurements instead of 64).

3. **Reconstruct:** Since we have fewer equations than unknowns, we use an optimization algorithm (like ISTA) to solve for x , imposing the sparsity constraint.

Advantage: We save hardware resources (sensors, bandwidth, storage) by never capturing the redundant data in the first place. Sparsity is the *Assumption* that makes the math work.

6 The Physical Meaning of $y = Ax$

The matrix A is not just a mathematical abstraction; it represents the physical sensing hardware.

6.1 Hardware Example: The Single Pixel Camera

Instead of a megapixel sensor, we can use a single photodiode.

- **The Hardware:** A Digital Micromirror Device (DMD) reflects light onto a single sensor.
- **The Matrix A :** Each row of A represents a specific pattern of mirrors (ON/OFF) displayed on the DMD.
- **The Measurement y_i :** The single sensor records the sum of light allowed by that specific mirror pattern.
- **The Process:** To get 10 measurements ($y \in \mathbb{R}^{10}$), we flash 10 different random patterns (rows of A) and record the intensity.

7 The Role of the FPGA

If the sensor (Hardware) performs the compression via A , the FPGA acts as the **Reconstruction Brain**.

1. **Input:** The FPGA receives the compressed measurement vector y (which looks like random noise to a human).
2. **Knowledge:** The FPGA has the dictionary matrix A (or its learned equivalent) “baked” into its memory/logic.

3. **Algorithm (LISTA):** The FPGA runs the iterative reconstruction algorithm (unrolled into layers) to solve the optimization problem.
4. **Output:** The FPGA produces the sparse code x (the actual image).

Summary: We put the matrix A into the hardware physics to save sensing power, and we use the FPGA to hallucinate the full image back into existence using math.